What are Agile practices?

Keeping the Code

Agile programming involves **delivering working, tested software** **every iteration** (two-four weeks). As the iterations flow, **this demand creates a new kind of pressure** as developers code more, modify code more, and stay focused on today's deadlines.

Under such production pressures, the teams' **traditional programming practices** (lengthy analysis design phases, testing at the end of the project, etc.) **will not be sufficient**. Agile best practices have proven to enable **more frequent delivery** **with higher quality**. These agile best practices help the **programmers and the code itself become more agile**. The smaller cycles of agile programming appear to some as less rigorous, but the effectiveness comes from the application of these practices with great discipline. This discipline leads to **extensible, low-defect code** with the **simplest robust design** **that will work** for the features currently implemented. It is well-factored and well-protected by unit tests.

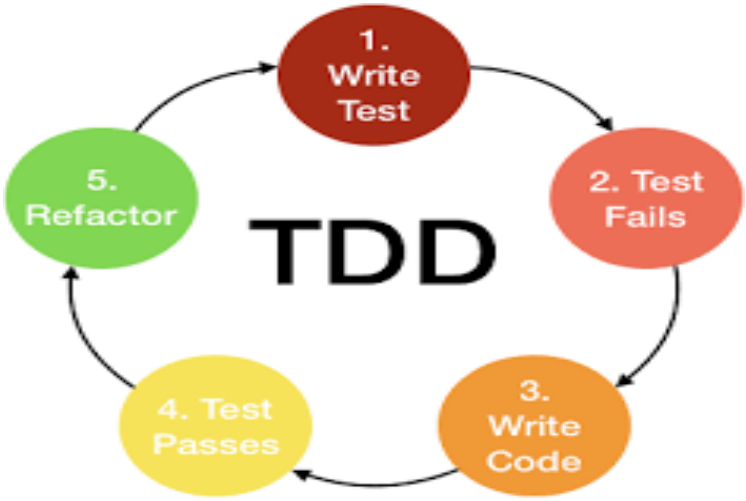
Best Practices Become Agile Software Programming

Long before we thought about agile software, programming teams were finding which **patterns** correlated to **greater success**. These patterns and practices have been proven over many decades at organizations writing some of industry's most complex software. First catalogued as Extreme Programming (XP), these practices have also come to be referred to as Agile Engineering Practices, Scrum Developer Practices, or simply Agile Programming.

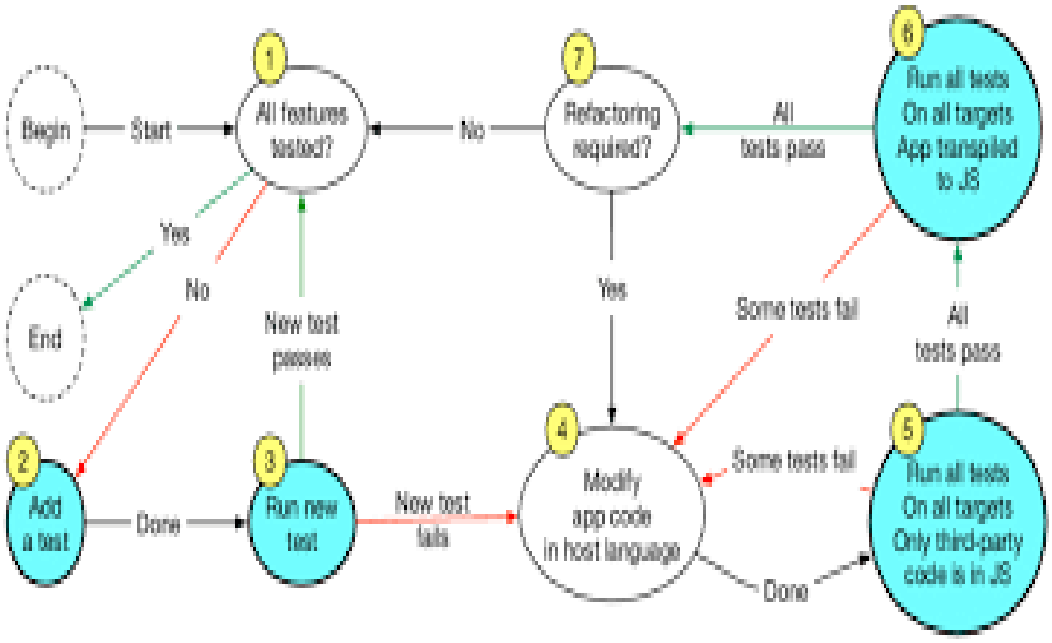
XP goes into the most depth concerning how programmers can keep themselves and their code agile. The XP practices have been embraced as enablers for all of the popular agile practices and lean approaches, including Scrum, SAFe, and Lean Startup. The community of developers passionate about these practices lives on in the Software Craftsmanship movement.

The core **agile software programming practices** are the following:

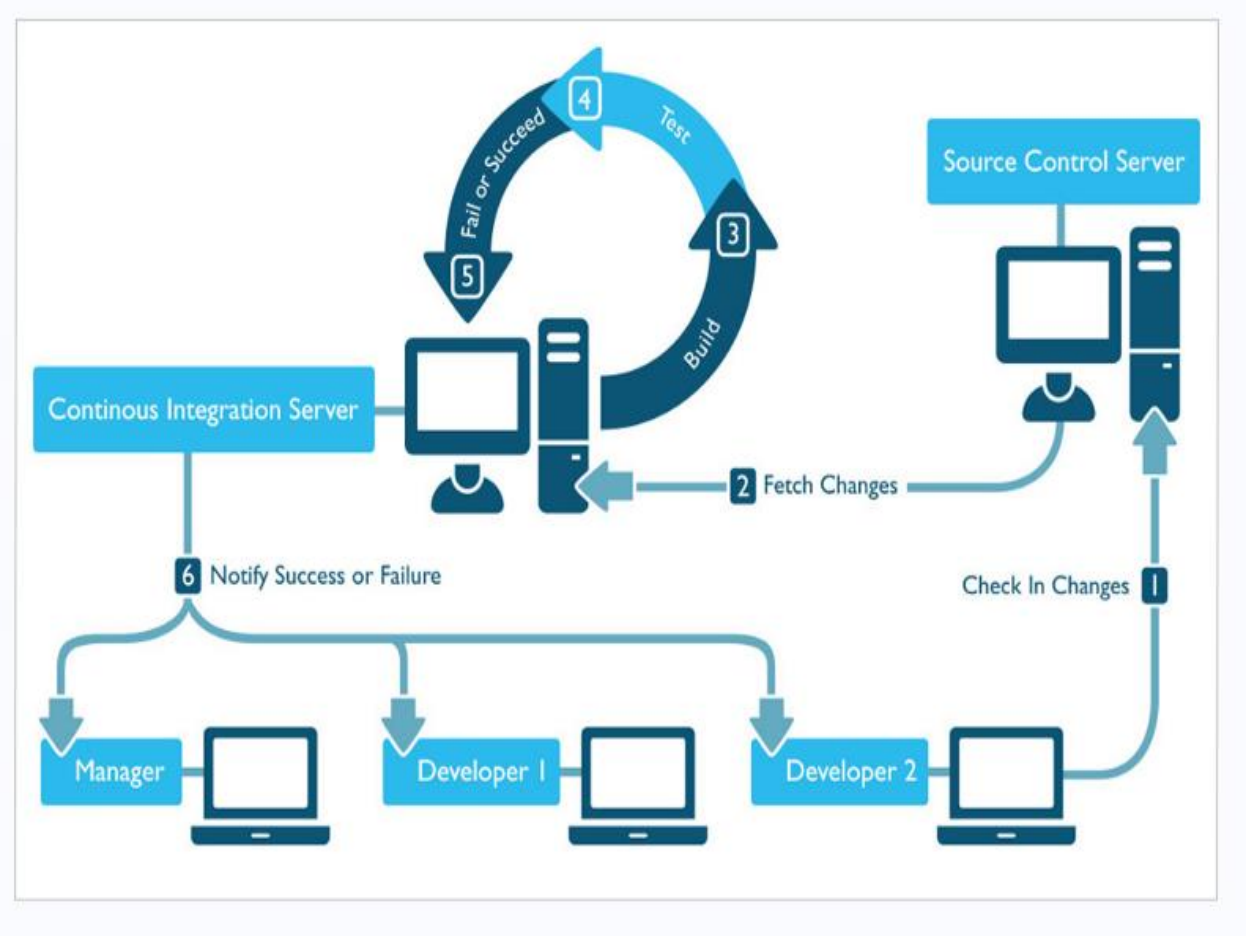
- Simple design
- Test-first programming (or perhaps Test-Driven Development)



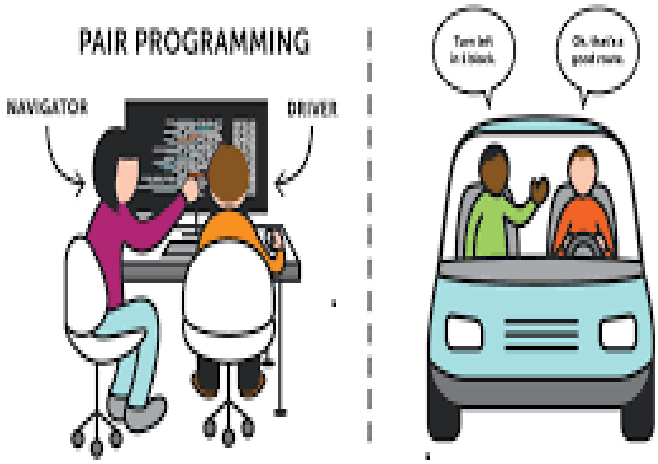
- Rigorous, regular refactoring



- Continuous integration



- Pair programming



- Sharing the codebase between all or most programmers

- A single coding standard to which all programmers adhere and

- A common "war-room" style work area.

Such practices provide the team with a kind of Tai Chi **flexibility**: a new feature, enhancement, or bug can come at the team from any angle, at any time, **without destroying** the project, the system, or production rates.